QA AUTOMATION LAB

Manual de arquitetura e aprendizado

Playwright + TypeScript
Da regra de negócio à execução no navegador

PORTFÓLIO QA / 29 OUT 2026

QA Automation Lab

Manual de arquitetura e aprendizado

Playwright + TypeScript | Automação Web com SauceDemo | Edição 29/10/2026

Este manual acompanha o código do portfólio. A proposta é entender o caminho completo de um cenário, desde a regra de negócio até o navegador. Leia com o projeto aberto e execute um caso por vez.

1. O que este projeto faz

A suíte controla um navegador real para validar uma loja de demonstração: entrar, ordenar produtos, adicionar e remover itens, informar dados fictícios e finalizar uma compra simulada. Um teste positivo verifica um fluxo permitido. Um teste negativo verifica como o sistema rejeita uma entrada inválida.

São 30 cenários: seis de autenticação, cinco de proteção de rotas, quatro de catálogo, quatro de detalhes, quatro de carrinho e sete de checkout. Os três motores desktop executam todos os cenários, e quatro cenários @smoke também executam no projeto mobile-chromium. São 94 execuções para 30 comportamentos distintos.

2. Arquitetura e responsabilidades

A suíte utiliza uma arquitetura em camadas para separar a intenção do teste dos detalhes de interação com a interface. Cada camada tem uma responsabilidade clara e pode evoluir sem espalhar mudanças pelos cenários.

• Tests: arquivos tests/*.spec.ts descrevem comportamentos e validam os resultados. Os identificadores permitem rastrear cada caso no plano e no relatório.

• Pages: src/pages contém classes por área da loja, com ações de negócio e localizadores tipados.

• Fixtures: src/fixtures/test.fixture.ts cria os objetos e prepara a autenticação em um contexto isolado por teste.

• Data: src/data/test-data.ts reúne usuários de demonstração, produtos e comprador fictício.

• Playwright: gerencia navegadores, ações, esperas automáticas e expectativas com repetição até o timeout.

• Configuração: playwright.config.ts define navegadores, execução paralela, evidências e relatórios.

• CI: GitHub Actions instala dependências, verifica tipos, executa a regressão e armazena artefatos.

O fluxo de leitura é teste, fixture, página e navegador. As expectativas de negócio ficam nos testes. A preparação autenticada verifica que o catálogo está pronto antes de entregar a página ao cenário. Localizadores permanecem próximos às ações que os utilizam, simplificando a manutenção.

3. Preparando a máquina

Instale Node.js 22.13 ou superior e abra o terminal na pasta que contém package.json. O arquivo .nvmrc indica a versão 22 para quem utiliza nvm. Execute os comandos abaixo na ordem:

```
npm ci
npx playwright install
npm run typecheck
npm run test:chromium
npm run report
```

npm ci instala exatamente as versões do package-lock.json. playwright install baixa os navegadores compatíveis com a versão instalada. No Linux, use npx playwright install --with-deps para instalar também as bibliotecas do sistema.

A execução exige internet para acessar o SauceDemo. As credenciais utilizadas são públicas e pertencem ao ambiente de demonstração. Nunca coloque credenciais reais no arquivo de dados.

4. Como ler um teste TypeScript

import traz uma função ou classe de outro arquivo. async marca uma função assíncrona. await aguarda uma operação como preencher um campo ou abrir uma página. Sem await, o teste pode terminar antes da ação ou executar passos fora da sequência esperada.

No parâmetro async ({ login, inventory, page }), as chaves selecionam fixtures por nome. page é a aba do navegador. login e inventory são objetos de página criados pela nossa fixture. test registra um cenário e expect verifica o resultado esperado.

```
await login.open();
await login.login(users.standard);
await expect(page).toHaveURL(/\/inventory\.html$/);
await expect(inventory.title).toHaveText('Products');
```

A expressão regular exige o caminho inventory.html no fim da URL. O ponto escapado significa um ponto literal. toHaveText aguarda o elemento atingir o texto esperado dentro do timeout. O teste não precisa de pausas fixas para aguardar a tela.

5. Camada de testes: o que validar

Abra tests/login.spec.ts. AUTH-01 demonstra um fluxo de autenticação: o comportamento continua sendo entrar com usuário válido e visualizar o catálogo. Os detalhes de preenchimento e clique foram movidos para LoginPage. As expectativas continuam no cenário, tornando claro o que significa passar.

Os casos negativos percorrem uma lista de objetos. Cada objeto informa identificador, título, usuário e mensagem esperada. O laço registra um teste independente por entrada; uma falha não impede os outros casos de serem registrados.

Os testes de checkout solicitam a fixture checkoutStarted para selecionar os produtos e abrir o formulário. A opção checkoutCart define os produtos: por padrão uma mochila; em CHK-07, dois itens. Cada cenário prepara seu próprio estado. O caso CHK-01 valida produto e valores antes de finalizar, e depois verifica a mensagem de conclusão.

6. Pages: como interagir com a tela

Uma classe agrupa dados e comportamentos. O constructor recebe Page, a aba controlada pelo Playwright. readonly impede reatribuir aquela propriedade no TypeScript. Não significa que o conteúdo da página seja imutável.

```
readonly username: Locator;

constructor(readonly page: Page) {
  this.username = page.getByTestId('username');
  // Os demais localizadores são inicializados aqui também.
}

async login(user: { username: string; password: string }) {
  await this.username.fill(user.username);
  await this.password.fill(user.password);
  await this.page.getByTestId('login-button').click();
}
```

getByTestId utiliza data-test porque playwright.config.ts configura testIdAttribute. Um Locator é uma descrição de como localizar o elemento; o Playwright resolve o elemento ao usar a ação. fill preenche um campo. click aciona o botão. selectOption escolhe uma opção de um select.

LoginPage concentra entrada e erro. InventoryPage concentra produtos, ordenação, acesso ao carrinho e logout. CartPage concentra itens e remoção. CheckoutPage concentra formulário, revisão e confirmação.

O método item encontra o cartão pelo nome do produto e restringe o botão àquele cartão. Isso evita selecionar o primeiro botão da página sem saber a qual produto pertence. A implementação usa texto literal exato para aceitar também nomes com parênteses, como no cenário PROD-04. ProductPage restringe os campos ao contêiner de detalhes para evitar confusão com os elementos do catálogo durante a navegação.

7. Fixtures: preparação e isolamento

src/fixtures/test.fixture.ts estende o test do Playwright com novos objetos. O tipo Fixtures define os nomes disponíveis e as classes que cada um entrega. A função use entrega o objeto ao teste e aguarda seu uso terminar.

```
login: async ({ page }, use) => {
  await use(new LoginPage(page));
};
```

A fixture authenticatedInventory depende de login, inventory e page. Ela abre a loja, entra com usuário válido, confirma a navegação e entrega o catálogo. Só é executada quando o teste a solicita ou depende dela.

Cada cenário usa seu próprio contexto de navegador. Cookies e armazenamento não são compartilhados entre testes. O navegador pode ser reaproveitado pelo runner, mas os contextos permanecem separados. Não usamos variáveis globais para controlar sessão e não fazemos login compartilhado em beforeAll.

8. Dados: o que muda sem mudar a regra

src/data/test-data.ts contém usuários, comprador fictício e produto. as const preserva tipos literais e expressa intenção de dados fixos. users.standard representa o usuário válido; locked e invalid são entradas para cenários negativos.

O produto escolhido tem nome e preço conhecidos. A revisão do checkout também compara subtotal, imposto e total esperados. Dados fixos são úteis aqui porque tornam o erro reproduzível. Dados aleatórios só devem ser adicionados quando houver uma necessidade de unicidade ou variedade controlada.

9. Configuração: como executar

playwright.config.ts define testDir, paralelismo, navegadores e evidências. fullyParallel permite que testes independentes executem em paralelo. workers limita quantos processos executam casos simultaneamente: três localmente e dois no CI.

forbidOnly impede publicar uma execução de CI incompleta por causa de test.only esquecido. retries vale zero localmente e dois no CI. Uma aprovação apenas na repetição indica instabilidade: failOnFlakyTests reprova o CI mesmo que um retry passe.

O timeout geral é de 30 segundos por teste e o de expect é de oito segundos. baseURL define a origem usada por page.goto('/'). BASE_URL pode sobrescrever o endereço via variável de ambiente. O projeto não carrega .env automaticamente.

Os projetos chromium, firefox e webkit representam os motores desktop. mobile-chromium usa o perfil Pixel 7 e filtra somente @smoke. Emula viewport, user agent e toque; não executa um aplicativo nativo nem comprova funcionamento em aparelho físico. No mobile, o botão dos detalhes pode exigir rolagem antes de aparecer na viewport; o teste faz essa rolagem e confirma que é alcançável.

10. Comandos para estudar

```
npm run test:list
npm run test:smoke
npx playwright test --project=chromium --grep AUTH-01
npm run test:ui
npm run test:debug
npm run report
```

Comece por AUTH-01. Use debug para avançar passo a passo e inspecionar os elementos. Depois rode AUTH-03 e observe a validação do erro. Por fim, execute CHK-01 e acompanhe as três etapas no relatório.

npm run typecheck executa tsc --noEmit. Playwright consegue transformar TypeScript para executar, mas isso não substitui a checagem completa de tipos. tsconfig.json ativa strict para detectar acessos e argumentos incompatíveis.

11. Relatórios e investigação de falhas

O reporter list mostra o progresso no terminal. HTML gera playwright-report e JUnit gera test-results/junit.xml. HTML é útil para leitura humana; JUnit pode ser consumido por ferramentas de CI.

Uma falha retém screenshot, vídeo e trace. O trace reúne ações, instantâneos e informações de rede úteis para investigar o ponto de quebra. Use npm run report e abra os anexos do cenário. Também pode usar npx playwright show-trace seguido do caminho real do arquivo trace.zip.

Se uma falha ocorrer antes de criar a página, não haverá screenshot ou vídeo útil. Primeiro veja a mensagem do processo. Navegador ausente pede instalação; erro de DNS pede investigação da rede; timeout num locator pede inspecionar a página e o seletor. Não aumente timeout nem acrescente pausas sem entender a causa.

12. Pipeline e GitHub

.github/workflows/playwright.yml executa em pushes e pull requests para main/master e aceita disparo manual. Instala dependências com npm ci, verifica lint, formatação e tipos, instala navegadores e executa npm test. O upload preserva relatório e resultados mesmo quando há falha de teste, exceto em cancelamento.

As permissões são limitadas à leitura do repositório. Os artefatos expiram após 14 dias. O workflow está preparado no código; publicar no GitHub e verificar uma execução real são passos separados. As cotas e cobranças de infraestrutura dependem do plano da conta.

Para publicar o portfólio, revise o diff e envie as alterações ao repositório da sua conta. Acompanhe a aba Actions para confirmar a instalação, a verificação de tipos e a execução dos testes no ambiente de CI.

13. Exercício: adicionar um cenário

CART-03 implementa persistência após recarregar. Leia seu código: ele adiciona o produto, chama page.reload, valida o badge e confirma nome e preço no carrinho. CART-04 mostra a remoção seletiva entre dois itens. CHK-05 e CHK-06 cancelam as etapas do checkout; CHK-07 verifica a soma de dois produtos e o carrinho vazio após concluir.

Como exercício novo, escreva CART-05: adicionar um produto, abrir o carrinho, voltar ao catálogo e confirmar que o botão do produto mostra Remove. Crie um método continueShopping em CartPage para acionar o botão de retorno. Valide também o badge. Execute o identificador isoladamente e atualize o plano e a contagem de testes.

Se precisar de uma ação nova de negócio, crie um método na página correspondente. Se precisar de dados novos, coloque-os no arquivo de dados.

Comandos para uma rotina profissional

npm run check verifica lint, formatação e tipos e executa Chromium. npm run quality executa somente as verificações estáticas. npm run format aplica a formatação definida pelo Prettier. O ESLint também detecta promises esquecidas sem await. npm run test:regression executa os três navegadores desktop e o smoke mobile Web. Os scripts test:login, test:catalog, test:cart e test:checkout permitem trabalhar por domínio. O guia docs/COMMANDS.md reúne exemplos de filtros e investigação.

Para ver o site sendo controlado ao vivo, execute npm run test:demo. Esse comando abre Chromium e insere 1,2 segundo entre ações na compra CHK-01. A janela fecha ao terminar. npm run test:headed abre o navegador em velocidade normal. npm run test:ui mostra o painel e snapshots das ações selecionadas. npm run test:debug permite avançar manualmente no Inspector.

14. Gerando novamente este PDF

O texto fonte é docs/MANUAL.md. scripts/generate_manual.py converte títulos, parágrafos, listas e blocos de código para um PDF com capa, cores e paginação. O arquivo de dependências Python está em scripts/requirements-docs.txt.

```
python3 -m venv .venv
.venv/bin/python -m pip install -r scripts/requirements-docs.txt
.venv/bin/python scripts/generate_manual.py
```

Se reportlab já estiver instalado no python3 ativo, npm run docs:pdf é suficiente. A instalação Python só é necessária para gerar documentação; não é necessária para executar testes.

15. Glossário e próximos passos

E2E: teste de ponta a ponta, pela interface. Assertiva: comparação entre resultado observado e esperado. Fixture: recurso e preparação entregues ao teste. Page Object: classe que representa ações de uma tela. Locator: descrição de um elemento. CI: execução automatizada após alterações. Smoke: seleção curta de fluxos essenciais. Flaky: teste cujo resultado oscila nas mesmas condições esperadas.

Os cenários de múltiplos produtos, cancelamento, persistência e ordenação inversa estão implementados nesta edição. Também estão cobertos detalhes de produto, proteção de rotas e smoke mobile Web. Como próximos passos, estude acessibilidade e API em uma aplicação que ofereça contrato documentado.

16. Referências públicas

• Page Objects: https://playwright.dev/docs/pom

• Fixtures e isolamento: https://playwright.dev/docs/test-fixtures

• Configuração: https://playwright.dev/docs/test-configuration

• Relatórios e trace: https://playwright.dev/docs/trace-viewer

• Aplicação de demonstração: https://www.saucedemo.com

17. Decisões, riscos e diagnóstico

O plano docs/TEST-PLAN.md relaciona riscos aos cenários que os cobrem. Leia docs/ARCHITECTURE.md para entender por que o checkout usa uma fixture parametrizável, por que o mobile executa somente smoke e por que o CI reprova instabilidade.

O arquivo docs/CASE-STUDY.md documenta um defeito real da automação: nomes com parênteses eram tratados como expressão regular. A correção usa correspondência literal e a regressão PROD-04 protege o comportamento. A reprodução mínima do erro de JavaScript está separada da execução real do teste.

Para estudar diagnóstico, use docs/TROUBLESHOOTING.md. As decisões de arquitetura e o estudo de caso mostram como relacionar riscos, código e evidências de execução.